

Documentazione di Progettazione Object-Oriented

Sistema Informativo per la Gestione di Campagne di Giochi di Ruolo (GDR)

Corso di Programmazione Object Oriented e Basi di Dati — A.A. 2025-2026

Svolto da: Pontillo Salvatore, Riccardi Carmine

Indice degli argomenti

Questo documento descrive la progettazione object-oriented del sistema realizzato per il progetto d'esame: l'architettura usata, le scelte fatte riguardo ai principi OOP, la gestione delle eccezioni e il funzionamento dei casi d'uso principali tramite i diagrammi di sequenza. Il documento è organizzato così:

- Sezione 1 — Introduzione e analisi del dominio applicativo
- Sezione 2 — Architettura del sistema (pattern BCE e DAO)
- Sezione 3 — Principi Object-Oriented e scelte progettuali
- Sezione 4 — Dizionario delle classi principali
- Sezione 5 — Gestione delle eccezioni
- Sezione 6 — Diagrammi di sequenza dei casi d'uso principali
- Sezione 7 — Diagramma delle classi UML del dominio

1. Introduzione e Analisi del Dominio

Il progetto consiste nella realizzazione di un sistema informativo per la gestione centralizzata di campagne di Giochi di Ruolo (GDR). Il dominio è stato pensato per due tipi di utenti:

- **Master:** è il creatore e l'arbitro della campagna. Può creare Razze, Classi e Oggetti, gestire i partecipanti e modificare le statistiche o l'inventario di un personaggio quando serve per la storia.
- **Giocatore:** è chi partecipa alla campagna. Può iscriversi alle campagne create dai Master, creare il proprio Personaggio Giocante (PG) rispettando le regole di classe e razza, gestire le proprie risorse (HP, Mana, Oro) e comprare oggetti nel negozio virtuale.

Il software realizzato offre tutti gli strumenti per poter visualizzare e modificare i dati relativi alle Campagne a cui ogni utente partecipa, omettendo invece la parte relativa alla gestione di un sistema di gioco vero e proprio basato su turni.

2. Architettura del Sistema: BCE e DAO

Il software è stato organizzato seguendo un'architettura a livelli, unendo il pattern BCE (Boundary-Control-Entity) con il pattern DAO (Data Access Object) per la persistenza dei dati. In questo modo i package sono più coesi al loro interno e meno legati tra loro, rendendo il sistema più facile da mantenere e modificare.

2.1. Pattern BCE

- **Boundary (package gui):** è l'interfaccia grafica, realizzata con Java Swing. Si occupa solo di mostrare i dati e raccogliere gli input dell'utente, senza contenere logica di business o accesso al database: si limita a chiamare il livello Control.
- **Control (package controller):** la classe Controller gestisce la logica dell'applicazione. Tiene lo stato della sessione (utente loggato, campagna attiva), controlla le richieste che arrivano dalla GUI e coordina le operazioni tra le Entity e il database.
- **Entity (package model):** rappresentano un'astrazione degli elementi di un gioco di ruolo, le strutture dati necessarie e gestiscono i dati transienti. Non dipendono né dall'interfaccia grafica né dal database.

2.2. Pattern DAO

Per la persistenza dei dati su PostgreSQL è stato usato il pattern DAO, organizzato in tre package:

- **dao:** contiene le interfacce (es. UtenteDAO, PersonaggioDAO, InventarioDAO) con le operazioni CRUD, senza dipendere da un DBMS specifico.
- **implementazionePostgresDAO:** contiene le classi che implementano queste interfacce per PostgreSQL (es. ImplementazionePostgresUtente, ImplementazionePostgresGiocatore, ImplementazionePostgresInventario).
- **database:** gestisce la connessione al database utilizzando il protocollo JDBC.

Grazie a questa divisione, per cambiare tecnologia di persistenza è sufficiente modificare le classi nel package dao.impl, senza dover toccare Controller ed Entity, rendendo il codice modulare ed estensibile.

3. Principi Object-Oriented e Scelte Progettuali

Il dominio è stato modellato usando i principi della programmazione a oggetti, puntando soprattutto sul riuso del codice e sulla protezione dei dati.

3.1. Ereditarietà e Polimorfismo

- **Gerarchia Utente:** è stata creata una classe astratta Utente con attributi comuni (username, password, email), da cui derivano le classi Giocatore e Master. Questo permette al Controller di usare un riferimento generico (`private Utente utenteAttivo;`) per tenere traccia dell'utente attivo nel sistema e di usare l'operatore `instanceof` per mostrare l'interfaccia grafica adatta al tipo di utente loggato.
- **Gerarchia Oggetto:** esiste una classe astratta Oggetto (id, nome, costo), da cui derivano OggettoConsumabile, con i parametri di cura, e OggettoEquipaggiabile, con requisiti e bonus statistici. La specializzazione in due classi distinte è resa necessaria dal comportamento diverso dei tipi di Oggetto, mentre la loro gestione polimorfica permette una gestione uniforme dell'inventario dei personaggi.

3.2. Incapsulamento e Protezione dello Stato

Gli attributi delle classi Entity sono privati e si accede a loro solo tramite getter e setter. Ad esempio, nella classe Personaggio l'oro e le risorse vitali non possono mai scendere sotto zero, e i metodi per curare il personaggio non fanno mai superare i valori massimi previsti dalle statistiche.

3.3. Riuso, Estensibilità e Separazione delle Responsabilità

- **Riuso del codice:** la classe Statistica è l'esempio principale. Invece di ripetere gli stessi attributi numerici (forza, destrezza, ecc.) in Personaggio, Razza e OggettoEquipaggiabile, questi sono raccolti in un'unica classe usata in contesti diversi, riducendo le ripetizioni.
- **Estensibilità:** le gerarchie Utente e Oggetto sono pensate per poter aggiungere in futuro nuove sottoclassi (es. un ruolo Osservatore o un nuovo tipo di oggetto) senza dover modificare il Controller, che lavora già sui tipi astratti. La gestione delle statistiche in tempo reale delle istanze della classe Personaggio, e la modellazione di attributi come punti salute e punti mana, sono stati progettati tenendo conto della possibile implementazione futura di un sistema di combattimento a turni coordinato dal Master.
- **Separazione delle responsabilità:** ogni livello dell'architettura (GUI, Controller, Entity, DAO) cambia solo per un motivo preciso, ad esempio la GUI cambia solo per motivi grafici e i DAO solo se cambia la persistenza. Questo rende più semplice sostituire un livello senza toccare gli altri.

4. Dizionario delle Classi Principali

Di seguito sono descritte le principali classi relative al package model.

Entity	Descrizione e responsabilita'
Utente, Giocatore, Master	La classe astratta Utente contiene le credenziali di accesso (username, password, email). Da essa derivano Master, che ha i permessi per creare e popolare il mondo di gioco, e Giocatore, che tiene lo storico delle iscrizioni e i Personaggi Giocanti (PG) creati, interagendo tramite questi ultimi con la Campagna istanziata dal Master.
Campagna	E' l'entità che raggruppa tutto il resto. Gestisce lo stato (Iniziata/Non Iniziata) per permettere o bloccare le iscrizioni, e contiene le liste di Giocatori che vi partecipano, Personaggi, Classi, Razze e Oggetti. Ogni Campagna ha un solo Master.
Personaggio	E' il cuore del gioco. Rappresenta l'alter-ego del Giocatore che si iscrive ad una Campagna. E' definito da Classe, Razza, una serie di statistiche che vengono calcolate a run-time attraverso metodi pubblici e privati come calcolaStatisticheFinali(). In alternativa, può essere un Personaggio non Giocante (PnG), i quali vengono creati e gestiti dal Master per popolare il mondo di gioco. Il tipo del Personaggio è definito dall'attributo isPg.
Classe e Razza	Definiscono le caratteristiche del personaggio. La Razza fornisce dei modificatori fissi alle statistiche di un Personaggio, che vengono applicate subito al momento della creazione. La Classe fornisce invece le abilità sbloccabili dal personaggio.
Statistica	E' la classe che maggiormente permette il riutilizzo del codice: invece di ripetere gli stessi attributi numerici (forza, destrezza, ecc.) in più classi, sono raggruppati qui e usati come profilo del Personaggio, modificatore della Razza e requisiti/bonus dell'OggettoEquipaggiabile. Per semplicità, due istanze con gli stessi attributi possono essere considerate diverse, poiché, ad esempio, due istanze diverse di Personaggio potrebbero avere le stesse statistiche.
Oggetto, OggettoConsumabile, OggettoEquipaggiabile	Gerarchia astratta (id, nome, costo) da cui derivano OggettoConsumabile, con i parametri di cura, e OggettoEquipaggiabile, con requisiti e bonus statistici. Grazie al polimorfismo le transazioni del negozio vengono gestite in modo uniforme.

5. Gestione delle Eccezioni

E' stato creato il package exception, che contiene una gerarchia di eccezioni checked personalizzate (es. AutenticazioneException, OggettoNonSelezionatoException, PersonaggioNonTrovatoException, NomePgNonValidoException).

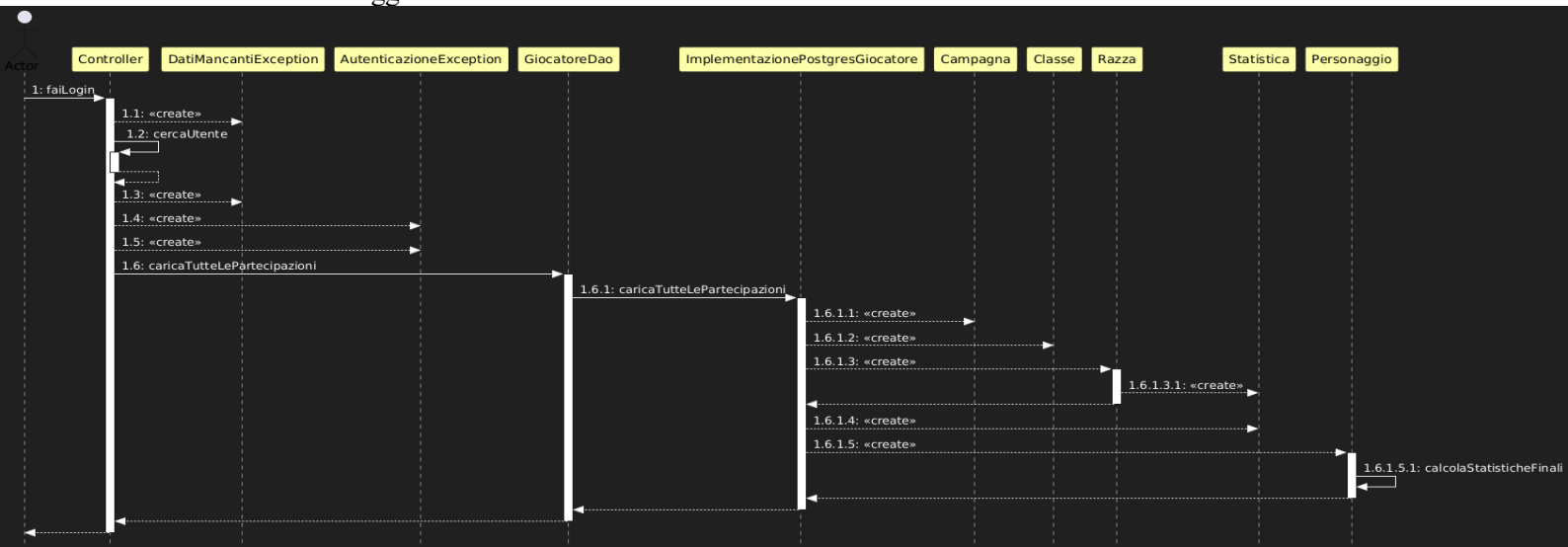
- **Gestione degli errori:** se una query viola i vincoli del database, il DAO intercetta la SQLException e lancia un'eccezione a runtime. Il Controller la gestisce e la trasforma in un messaggio comprensibile per la GUI, senza mai mostrare all'utente errori tecnici come SQLException o NullPointerException.
- **Transazioni:** nelle operazioni più delicate, come l'acquisto di un oggetto, l'autocommit viene disabilitato. Se qualcosa va storto durante l'operazione, viene eseguito un rollback() per lasciare il database in uno stato coerente.

6. Diagrammi di Sequenza dei Casi d'Uso

Per mostrare come funzionano le operazioni più importanti attraverso i vari livelli (BCE + DAO), sono stati realizzati i seguenti diagrammi di sequenza.

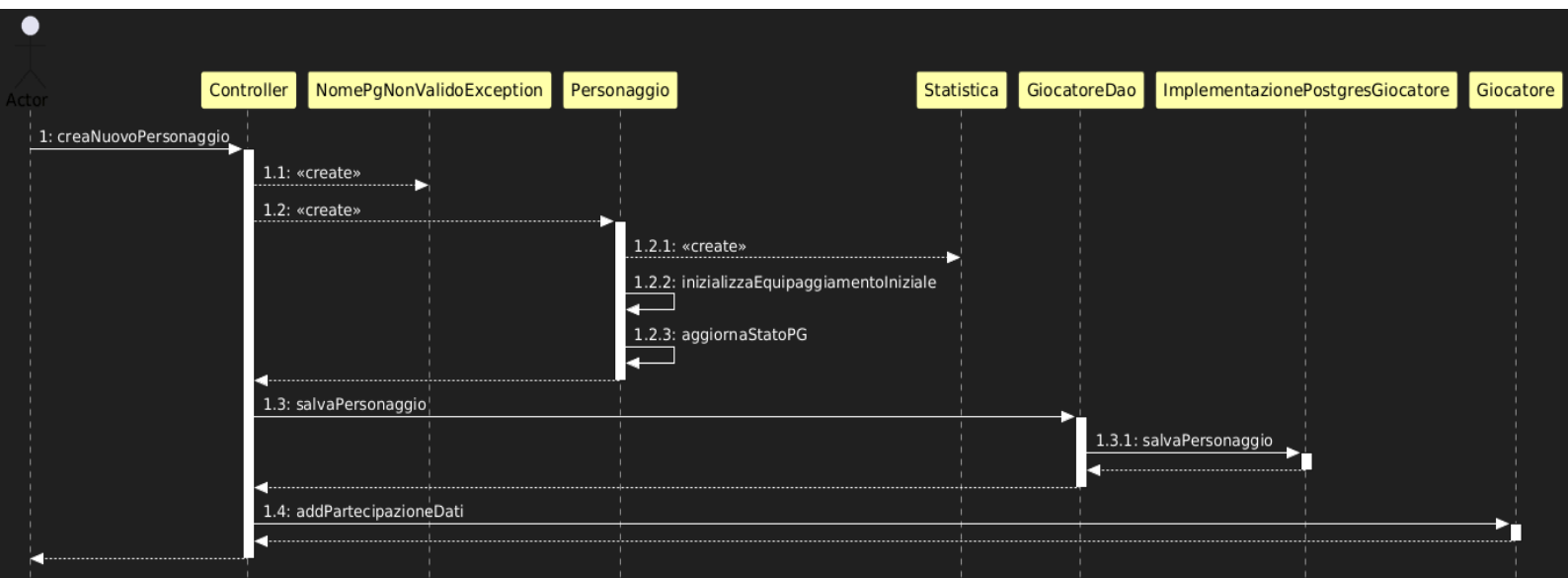
6.1. Autenticazione (faiLogin)

Il diagramma mostra il processo di login. Il Controller verifica le credenziali; se l'utente è un Giocatore, il sistema interroga il GiocatoreDao per recuperare le campagne a cui è iscritto, caricando anche i relativi Personaggi Giocanti e ricalcolandone le statistiche finali.



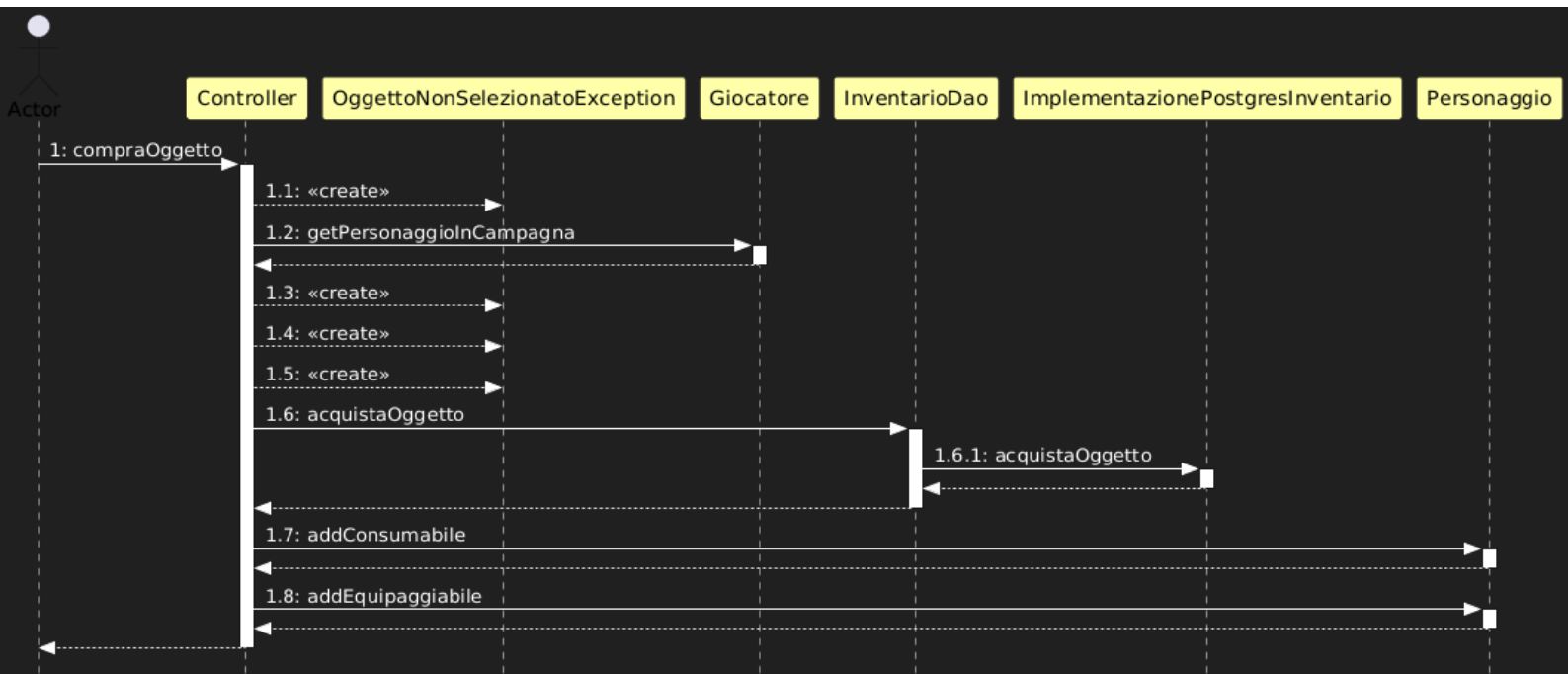
6.2. Creazione Personaggio (creaNuovoPersonaggio)

Mostra cosa succede quando un giocatore crea il primo personaggio in una campagna. Il Controller crea un nuovo Personaggio con l'equipaggiamento di base, lo salva tramite il GiocatoreDao e associa l'istanza creata allo stato del giocatore in memoria.



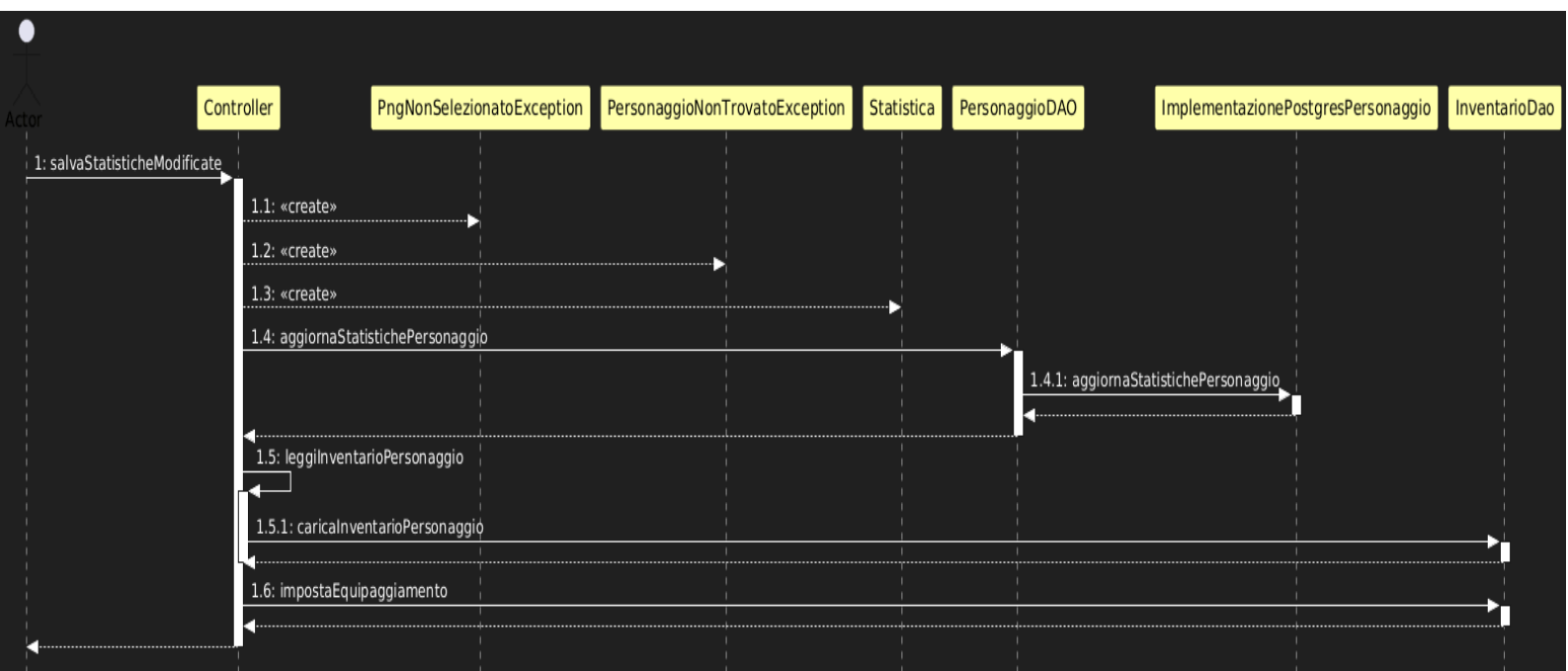
6.3. Acquisto Oggetto (compraOggetto)

Mostra la transazione economica nel negozio. Il Controller controlla che il giocatore abbia abbastanza oro e che non possieda già l'oggetto; se i controlli vanno a buon fine, viene chiamato l'InventarioDao che esegue l'operazione sul database e solo dopo l'oggetto viene aggiunto all'inventario in memoria.



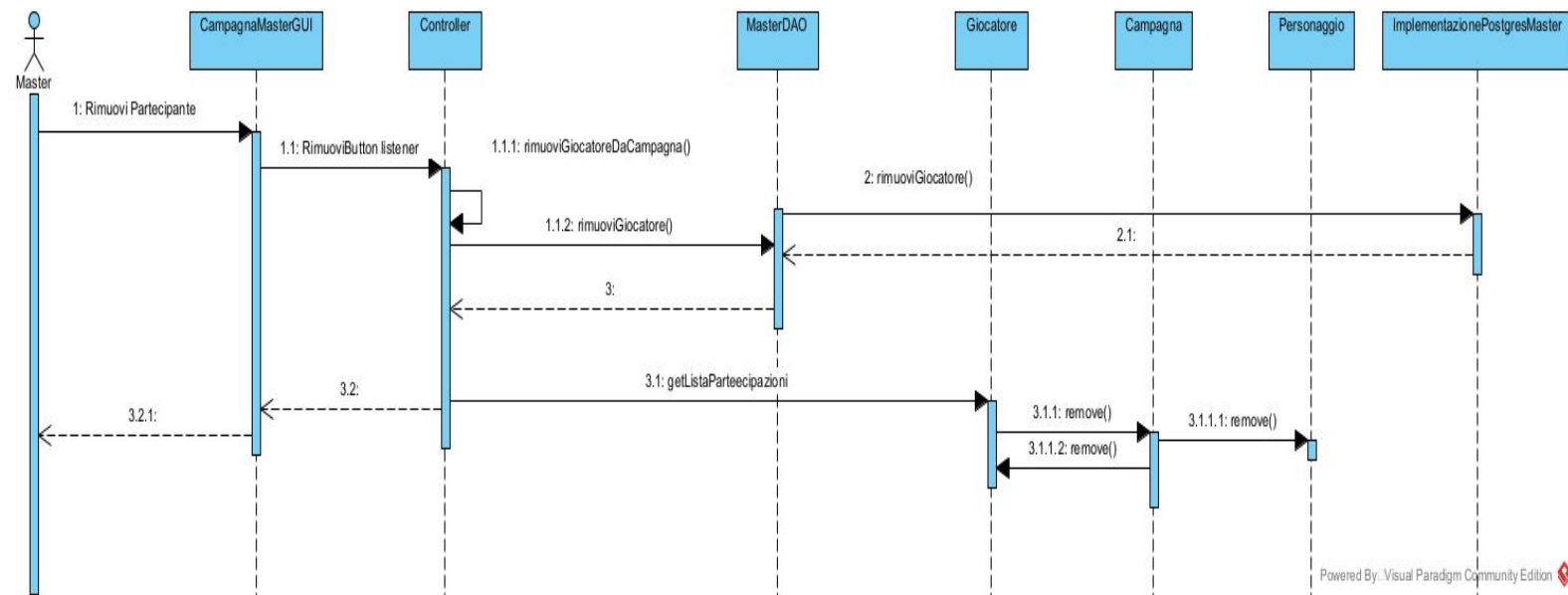
6.4. Override Statistiche Master (salvaStatisticheModificate)

Mostra un'operazione riservata al Master. Quando le statistiche di un PG vengono forzate manualmente, il Controller richiede l'aggiornamento al PersonaggioDAO e poi ricontrolla l'inventario: se le nuove statistiche non soddisfano più i requisiti dell'equipaggiamento indossato, l'oggetto viene tolto automaticamente tramite l'InventarioDao.



6.5 Rimozione Giocatore da Campagna (rimuoviPartecipante)

Mostra un'operazione riservata al Master. Consiste nella rimozione forzata di un Giocatore dalla Campagna gestita dal Master; il listener del button associato in CampagnaMasterGUI chiama il controller, che richiede prima l'eliminazione a MasterDAO. Quest'ultimo tenta di eliminare dal database i dati relativi al Giocatore, compreso il Personaggio ad esso associato e tutti i dati relativi al Personaggio. Se l'operazione va a buon fine, il Controller chiama il model per aggiornare i dati in memoria.



7. Diagramma delle Classi UML del Dominio

Il diagramma seguente mostra le relazioni tra le classi del dominio (associazioni, aggregazioni, composizioni e generalizzazioni).

